TD PRATIQUE

Git, DVC & Segmentation avec YOLOv8

Versionnage de code · Versionnage de données · Inférence IA sur images de nourriture

---

Environnement : Windows 10/11 — PowerShell — VS Code

Prérequis : Python 3.10+ · Compte GitHub · Compte Google

Dataset : fourni par l'enseignant — 100 images de nourriture

CONTEXTE ET OBJECTIFS

Dans ce TD, tu vas construire de A à Z un **mini-pipeline MLOps** autour d'un projet de segmentation d'images de nourriture. L'objectif n'est pas d'obtenir un modèle IA performant, mais de mettre en place les bonnes pratiques professionnelles qui entourent tout projet de Machine Learning sérieux.

Le projet

Tu disposes d'un dataset de **100 images de nourriture** (plats variés). Tu vas appliquer le modèle de segmentation **YOLOv8m** sur ces images pour détecter et délimiter automatiquement les différents éléments présents dans chaque plat — bol, assiette, ustensiles, aliments identifiables, etc.

Ce type de pipeline est directement applicable à des systèmes réels d'analyse nutritionnelle à partir d'images, comme des applications mobiles qui estiment la composition d'un repas photographié.

Pourquoi Git, DVC et YOLOv8 ?

Tout projet IA en production repose sur trois piliers que ce TD te fait pratiquer :

- **Git** — versionner le code. Sans Git, il est impossible de tracer qui a modifié quoi, de revenir en arrière après une erreur, ou de collaborer à plusieurs sans se marcher dessus. C'est le minimum absolu dans tout projet informatique.
- **DVC (Data Version Control)** — versionner les données et les modèles. Un modèle IA dépend autant de son code que de ses données d'entraînement. Si les données changent sans être tracées, les résultats deviennent non reproductibles. DVC applique les mêmes principes que Git aux fichiers lourds, en les stockant sur Google Drive.
- **YOLOv8m** — le modèle IA. Il te donne un cas concret et réaliste pour pratiquer l'intégration d'un modèle dans un pipeline versionné. On utilise le modèle pré-entraîné — pas besoin de GPU ni de phase d'entraînement.

Ce que tu auras à la fin

À l'issue de ce TD, tu disposeras d'un dépôt GitHub professionnel contenant :

- L'historique complet de tes modifications de code (Git)
- Tes 100 images versionnées et synchronisées sur Google Drive (DVC)
- Un script de segmentation fonctionnel produisant des images annotées (YOLOv8m)
- Un modèle téléchargé et versionné séparément via DVC

Organisation du TD

Partie	Contenu	Outils
Partie 1	Versionner le code et publier sur GitHub	Git, GitHub
Partie 2	Versionner les données sur Google Drive	DVC, Google Cloud
Partie 3	Segmenter les images de nourriture	Python, YOLOv8m

> *Ce TD est un point de départ vers le MLOps. Les compétences pratiquées ici — versionnage de code, versionnage de données, reproductibilité — sont les fondations de tout pipeline de Machine Learning en environnement professionnel.*

PARTIE 1 — GIT : Versionner son code

Git est un système de contrôle de version distribué. Il enregistre l'historique complet de ton code sous forme de *commits*, te permet de revenir à n'importe quelle version antérieure, et facilite la collaboration en équipe via des plateformes comme GitHub. Dans cette partie, tu vas initialiser un dépôt local, publier ton code sur GitHub, puis pratiquer le travail en branches.

Tâche 1 — Configurer Git

Avant toute chose, Git a besoin de connaître ton identité. Cette information est attachée à chaque commit que tu créeras, ce qui permet d'identifier l'auteur de chaque modification dans l'historique du projet.

Ouvre VS Code, puis accède au terminal intégré via **Terminal** → **Nouveau terminal**.

```
# Vérifier que Git est correctement installé

git --version


# Enregistrer ton nom (apparaîtra dans l'historique)

git config --global user.name "Ton Nom"


# Enregistrer ton email (doit correspondre à ton compte GitHub)

git config --global user.email "ton@email.com"


# Vérifier la configuration enregistrée

git config --list
```

Tâche 2 — Créer le projet et initialiser Git

Tu vas créer le dossier racine du projet et demander à Git de le prendre en charge. La commande **git init** crée un sous-dossier caché **.git/** qui contient l'intégralité de la base de données de versionnage.

```
# Créer le dossier du projet

mkdir food-segmentation

cd food-segmentation
```

```
# Ouvrir le dossier dans VS Code

code .


# Initialiser le dépôt Git local

git init
```

Tâche 3 — Créer la structure du projet et faire le premier commit

Un projet bien structuré dès le départ facilite la maintenance et la collaboration. Suis chaque étape dans l'ordre.

Étape 3.1 — Créer l'arborescence

Sous Windows PowerShell, la commande **touch** n'existe pas. On utilise **New-Item** pour créer des fichiers vides.

```
# Créer les répertoires du projet

mkdir -p data/images # stockage des images du dataset

mkdir -p models # stockage des modèles IA

mkdir -p src # code source Python


# Créer les fichiers de base du projet

New-Item src/segment.py # script principal de segmentation

New-Item README.md # documentation du projet

New-Item .gitignore # liste des fichiers exclus du versionnage
```

Ouvre **.gitignore** dans VS Code et colle le contenu suivant. Ce fichier est essentiel : il indique à Git les fichiers et répertoires à ne jamais versionner (fichiers temporaires, modèles binaires volumineux, environnements virtuels).

```
__pycache__/

*.pyc

venv/

.env

*.pt
```

```
*.pth

.DS_Store

Thumbs.db
```

> *Sans le .gitignore, un modèle de 200 Mo pourrait être committé par erreur, rendant le dépôt inutilisable. C'est une bonne pratique systématique dans tout projet ML.*

Étape 3.2 — Indexer les fichiers et créer le premier commit

Le workflow Git suit toujours le même cycle : **modifier** → **indexer (add)** → **sauvegarder (commit)**. L'indexation te permet de choisir précisément quels fichiers inclure dans chaque commit.

```
# Inspecter l'état du dépôt (fichiers en rouge = non suivis)

git status


# Indexer l'ensemble des fichiers (staging)

git add .


# Vérifier l'index (fichiers en vert = prêts à être commités)

git status


# Créer le premier commit

# Convention de message : "type: description courte"

git commit -m "init: structure du projet"


# Consulter l'historique des commits

git log --oneline
```

Tâche 4 — Publier le dépôt sur GitHub

GitHub est une plateforme d'hébergement de dépôts Git. Elle joue le rôle de serveur central (remote) à partir duquel toute l'équipe peut synchroniser son travail.

- Va sur **github.com** et connecte-toi à ton compte
- Clique sur **+** (haut à droite) → **New repository**

- Nom du dépôt : **food-segmentation**
- Laisse tous les paramètres par défaut (pas de README, pas de .gitignore)
- Clique sur **Create repository**

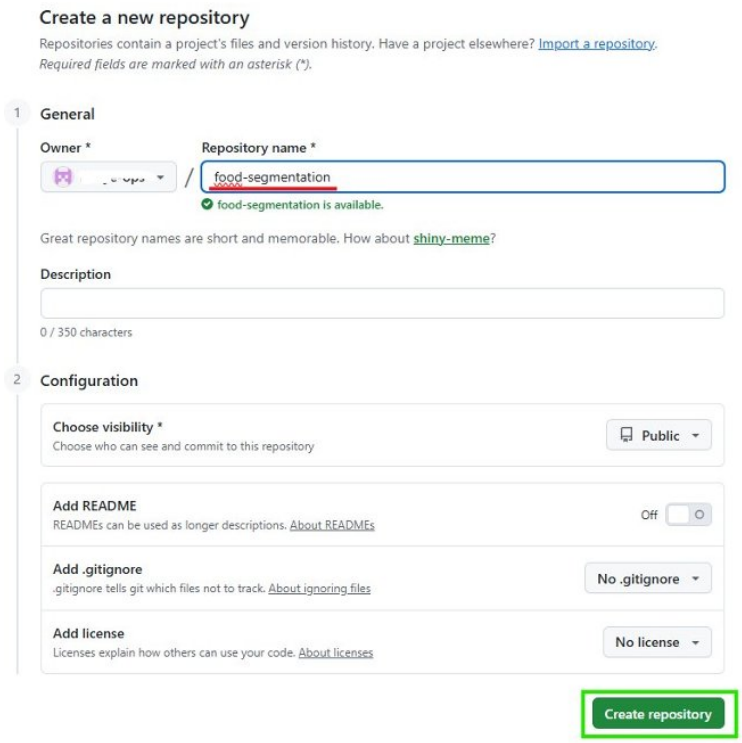


Fig. 1 — Création du dépôt food-segmentation sur GitHub

GitHub affiche ensuite une page d'instructions. Utilise exclusivement la section **"...or push an existing repository from the command line"** encadrée ci-dessous, puisque ton dépôt local existe déjà :

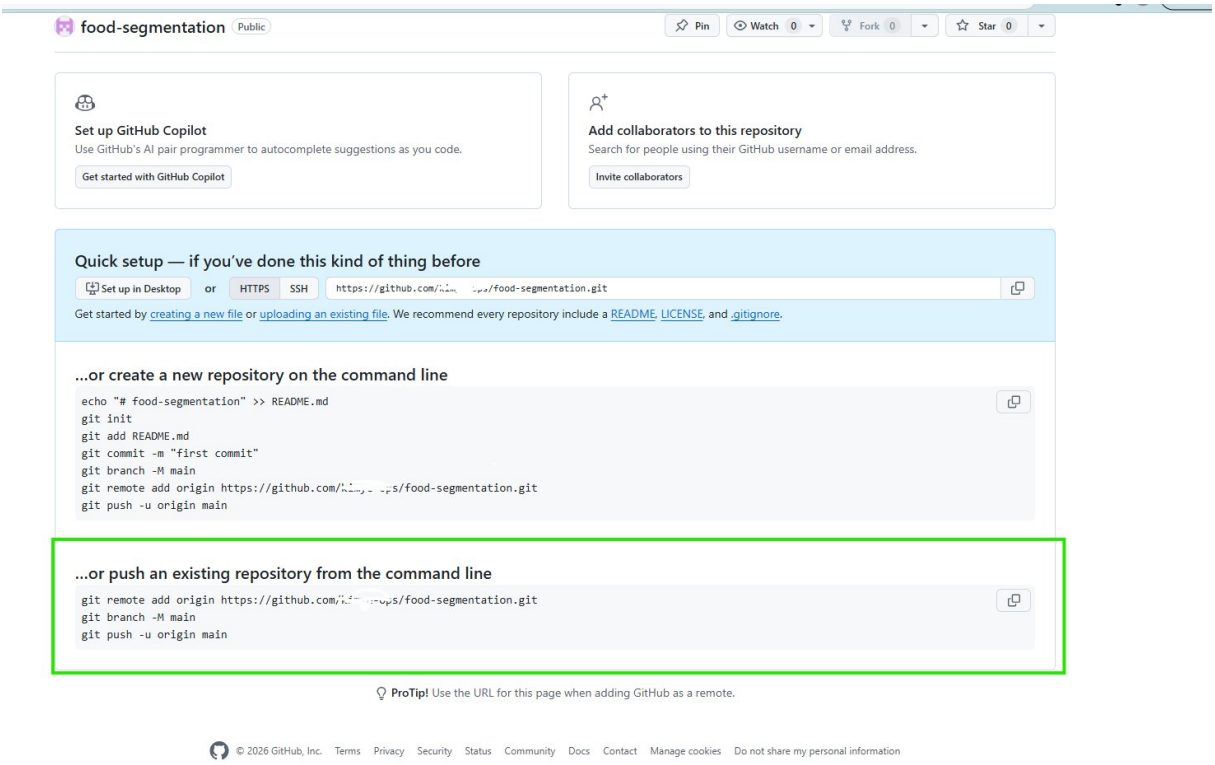


Fig. 2 — Page GitHub après création : utilise la section encadrée en vert

```
# Déclarer le dépôt GitHub comme remote "origin"

git remote add origin https://github.com/TON_USERNAME/food-segmentation.git


# Renommer la branche principale en "main" (convention GitHub actuelle)

git branch -M main


# Pousser le code vers GitHub et définir origin/main comme remote par défaut

git push -u origin main
```

Tâche 5 — Travailler avec les branches

Une branche est une ligne de développement parallèle et isolée. Elle te permet de travailler sur une fonctionnalité ou une correction sans affecter la branche principale **main**. Une fois le travail validé, tu fusionnes la branche dans main via un **merge**.

```
# Créer la branche et basculer dessus immédiatement

git checkout -b feature/dvc-setup
```

```
# Modifier le README pour documenter les outils du projet

"## Outils utilises" >> README.md

"- Git" >> README.md

"- DVC" >> README.md

"- YOLOv8" >> README.md


# Indexer et commiter sur la branche

git add README.md

git commit -m "docs: ajout section outils dans README"


# Revenir sur main et fusionner la branche

git checkout main

git merge feature/dvc-setup

git push origin main


# Nettoyer : supprimer la branche locale devenue inutile

git branch -d feature/dvc-setup
```

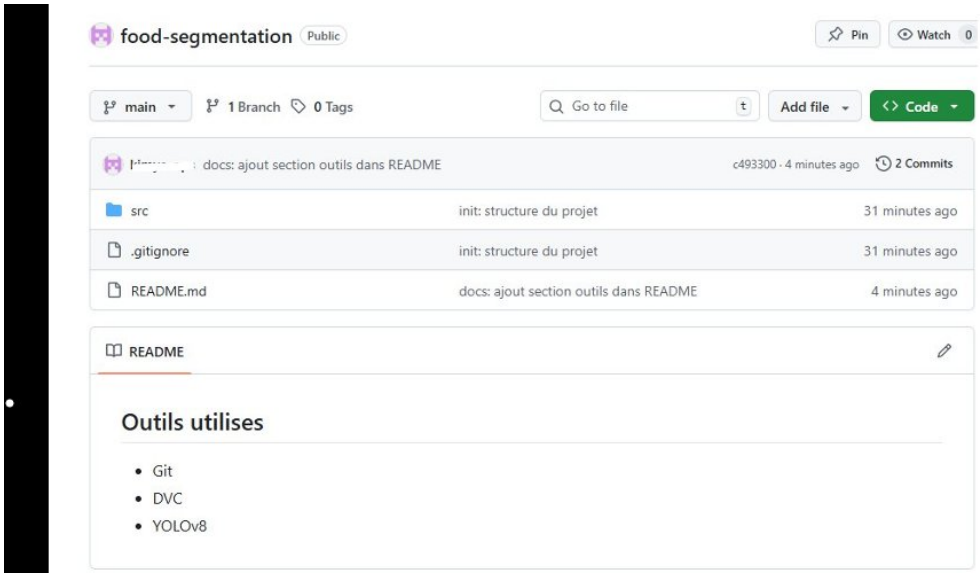


Fig. 3 — Résultat sur GitHub : 2 commits, branche main, README à jour

Workflow Git fondamental à mémoriser : git add → git commit → git push. Ces trois commandes constituent le cycle de base que tu répéteras à chaque modification de code.

PARTIE 2 — DVC : Versionner les données

Git est optimisé pour les fichiers texte de petite taille. Il n'est pas conçu pour gérer des assets volumineux tels que des datasets d'images ou des modèles entraînés. **DVC (Data Version Control)** comble ce manque : il applique les mêmes principes que Git aux données, en déléguant le stockage physique à un remote externe comme Google Drive. Dans le dépôt Git, seul un fichier pointeur **.dvc** (quelques lignes de texte) est versionné.

```
Code source ■■■■ Git ■■■■■ GitHub (fichiers légers)

Données / IA ■■■■ DVC ■■■■■ Google Drive (fichiers lourds)
```

Tâche 1 — Installer DVC

```
pip install dvc dvc-gdrive

dvc --version
```

Tâche 2 — Initialiser DVC dans le projet

DVC doit être initialisé à la racine du dépôt Git. Il crée son propre répertoire de configuration **.dvc/**, que l'on versionne ensuite dans Git.

```
dvc init

git add .dvc .dvcignore

git commit -m "init: configuration DVC"

git push origin main
```

Tâche 3 — Ajouter les images du dataset

Décompresse l'archive fournie par l'enseignant et place les 100 images dans le répertoire **data/images/**.

```
# Compter les fichiers présents (PowerShell)

(ls data/images/).Count # doit retourner 100
```

Tâche 4 — Configurer Google Drive comme remote DVC

Suis chaque étape dans l'ordre. Les captures ci-dessous correspondent exactement à ce que tu verras sur console.cloud.google.com.

Étape A — Créer un projet Google Cloud

Google Cloud est la plateforme qui gère les autorisations d'accès à ton Google Drive via des identifiants OAuth. Tu dois y créer un projet dédié pour ce TD.

- Va sur **console.cloud.google.com**
- Clique sur **Sélectionner un projet** → **Nouveau projet**
- Nom : **dvc-td** → **Créer**

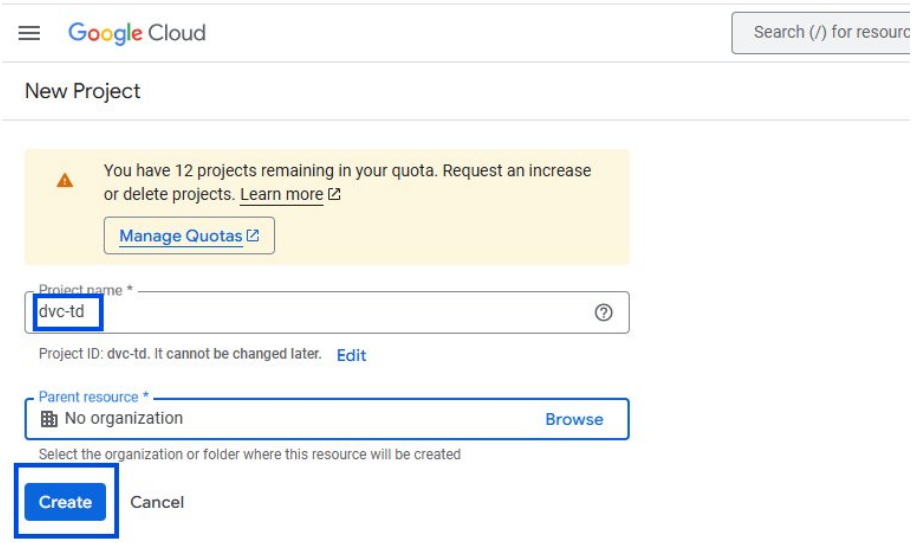


Fig. 4 — Création du projet dvc-td dans Google Cloud Console

Étape B — Activer l'API Google Drive

L'API Google Drive doit être explicitement activée sur ton projet avant de pouvoir l'utiliser.

- Menu gauche : **API et services** → **Bibliothèque**

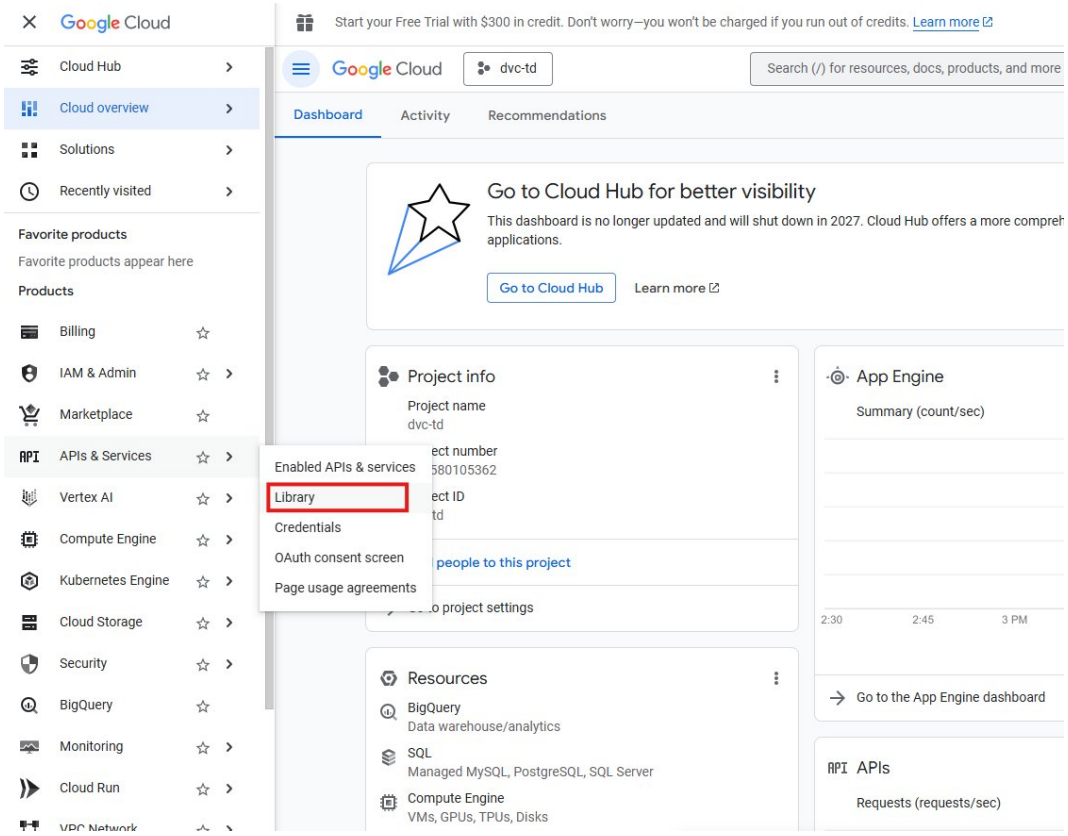


Fig. 5 — Navigation vers la bibliothèque d'API

• Recherche **Google Drive API** → clique dessus → **Activer**

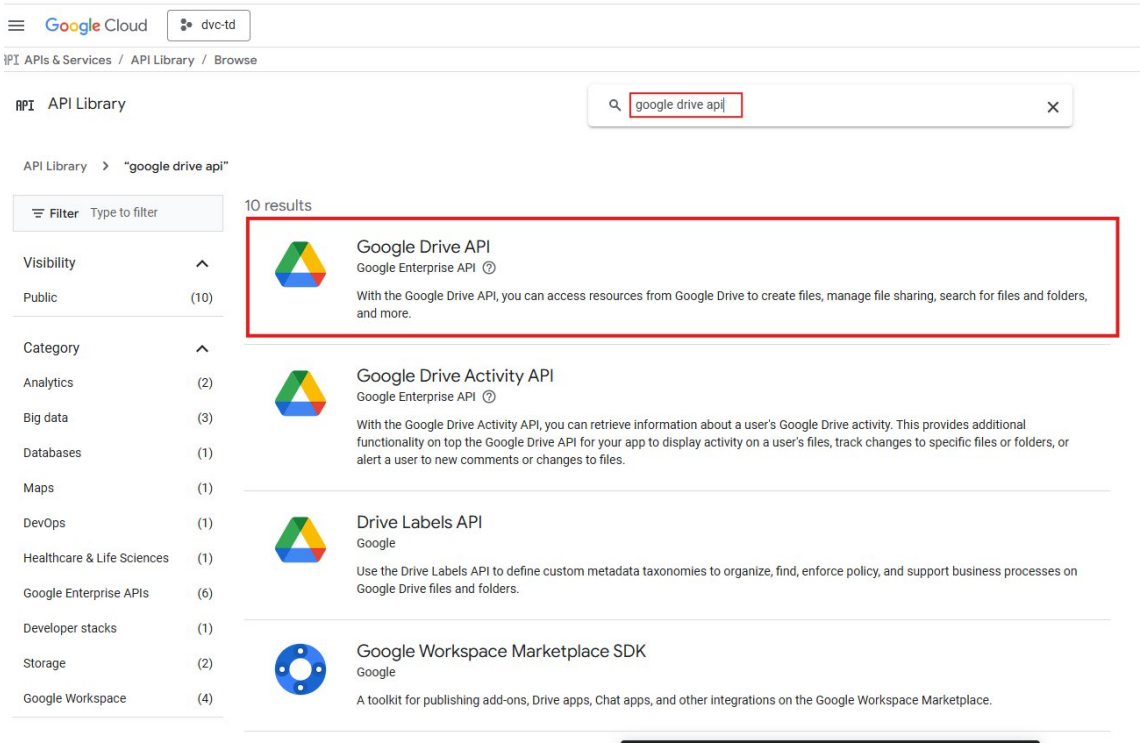


Fig. 6 — Activation de Google Drive API

Étape C — Créer les identifiants OAuth 2.0

Les identifiants OAuth permettent à DVC de s'authentifier auprès de Google Drive en ton nom, sans avoir à partager ton mot de passe.

- **API et services → Identifiants → + Créer des identifiants → ID client OAuth**
- Sélectionne **Google Drive API** et **Données utilisateur**

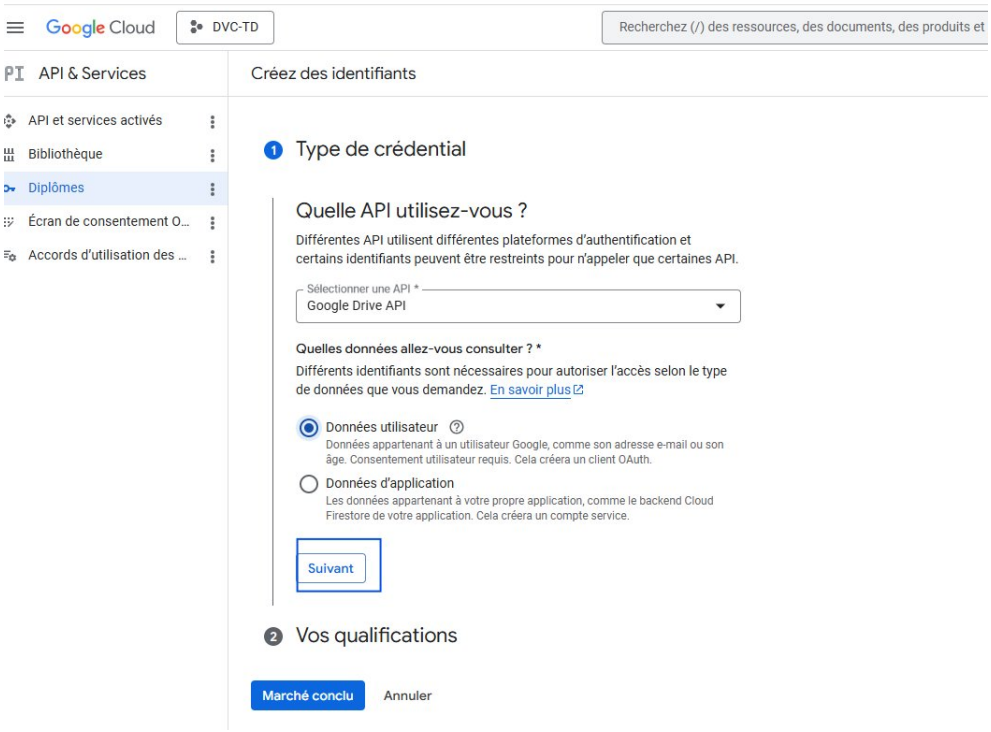


Fig. 7 — Sélection du type d'identifiants

- Remplis le formulaire de consentement : **Nom de l'application** : dvc-client · **Email de contact** : ton email Google · clique sur **Sauvegarder et continuer**

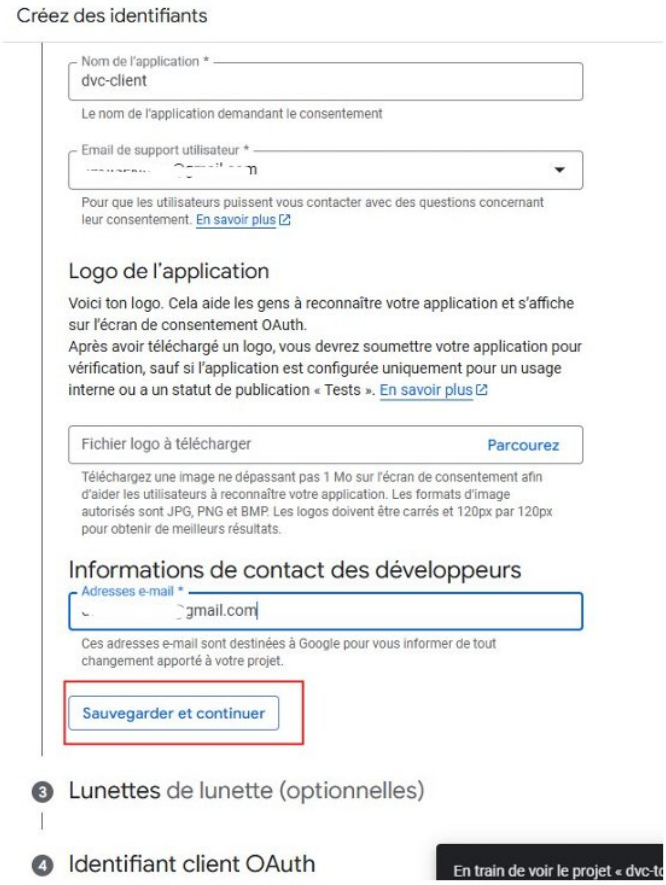


Fig. 8 — Formulaire de consentement OAuth : nom, email, puis Sauvegarder et continuer

• Type d'application : **Application de bureau** → **Créer**

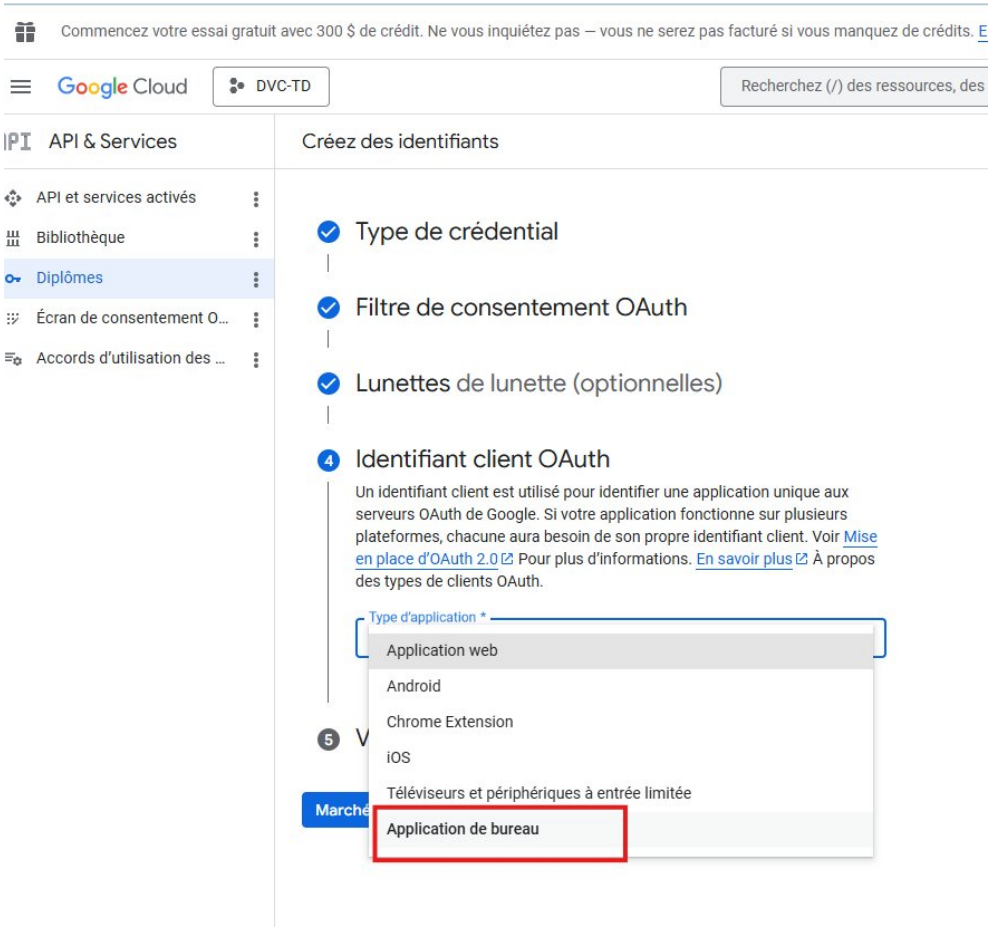


Fig. 9 — Type d'application : Application de bureau

- Copie le **Client ID** et le **Client Secret** affichés — tu en auras besoin à l'étape E

Étape D — Publier l'application OAuth (évite les blocages)

Par défaut, une application OAuth nouvellement créée est en mode **Test**, ce qui restreint l'accès aux seuls testeurs enregistrés. Pour permettre une authentification fluide lors du **dvc push**, publie l'application en mode Production.

- **API et services** → **OAuth consent screen**
- Section **Audience** → clique sur **Publish app**
- Confirme la publication dans la boîte de dialogue

Étape E — Créer le dossier Drive et connecter DVC

- Va sur **drive.google.com** → crée un dossier **dvc-food-data**
- Ouvre-le → copie l'identifiant dans l'URL :
  drive.google.com/drive/folders/CECI_EST_L_ID

```
# Déclarer le dossier Google Drive comme remote DVC par défaut
```

```
dvc remote add -d gdrive_remote gdrive://VOTRE_ID_DOSSIER

# Injecter tes identifiants OAuth dans la configuration DVC
dvc remote modify gdrive_remote gdrive_client_id 'VOTRE_CLIENT_ID'
dvc remote modify gdrive_remote gdrive_client_secret 'VOTRE_CLIENT_SECRET'

# Versionner la configuration dans Git
git add .dvc/config
git commit -m "config: remote DVC Google Drive"
git push origin main
```

Tâche 5 — Versionner les images et synchroniser avec Drive

```
# Placer le dossier images/ sous surveillance DVC
# → génère data/images.dvc (pointeur) et met à jour data/.gitignore
dvc add data/images

# Versionner le fichier pointeur dans Git (pas les images elles-mêmes)
git add data/images.dvc data/.gitignore
git commit -m "data: ajout dataset 100 images (v1)"
git push origin main

# Transférer les images vers Google Drive
dvc push
```

Lors du **dvc push**, DVC ouvre ton navigateur pour l'authentification Google. La page suivante s'affiche — c'est un avertissement de sécurité standard pour les applications non encore vérifiées par Google. Clique sur **Continue** pour autoriser l'accès :

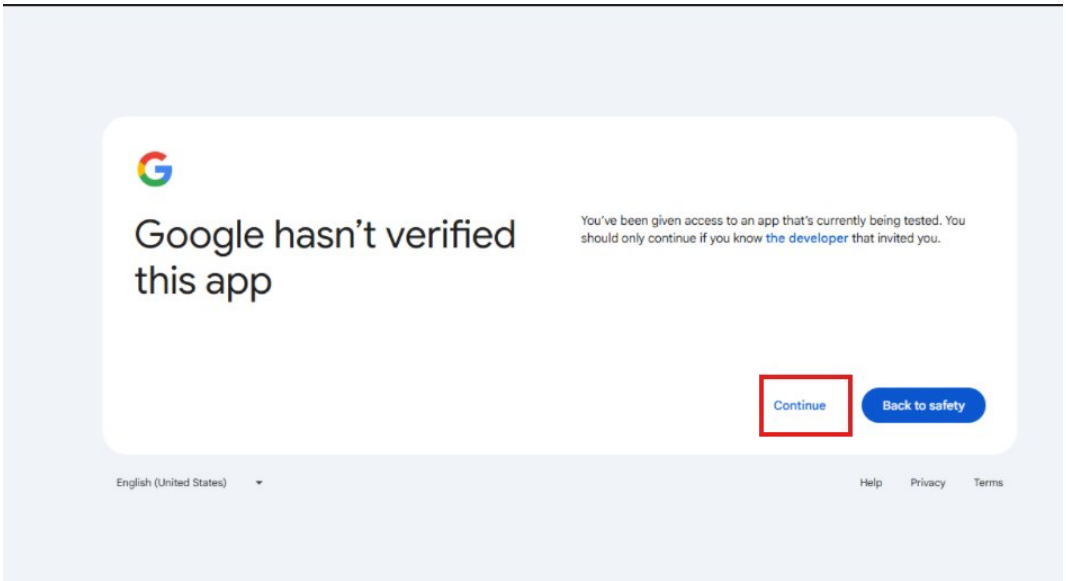


Fig. 10 — Avertissement Google : clique sur Continue pour poursuivre

Une fois l'autorisation accordée, le navigateur affiche le message de confirmation ci-dessous. Tu peux fermer l'onglet et revenir dans le terminal.

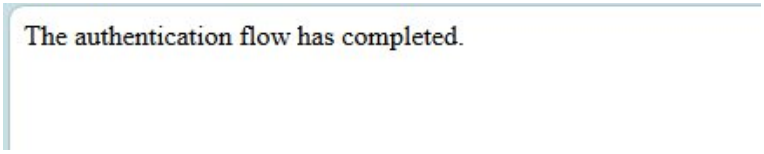


Fig. 11 — Authentification réussie : le transfert DVC reprend automatiquement

Tâche 6 — Tester la récupération des données (dvc pull)

Cette tâche simule le scénario où un nouveau collaborateur clone le dépôt et doit récupérer les données depuis Google Drive.

```
# Supprimer les images locales pour simuler un clone vierge

Remove-Item -Recurse -Force data/images


# Récupérer les données depuis Google Drive via DVC

dvc pull


# Vérifier la récupération

(ls data/images/).Count # doit retourner 100
```

Workflow DVC au quotidien : dvc add (nouvelles données) → git commit → dvc push | dvc pull pour récupérer les données sur un autre poste.

PARTIE 3 — YOLOv8m : Segmenter les images

YOLOv8m (You Only Look Once — version 8, taille medium) est un modèle de vision par ordinateur développé par Ultralytics. Dans sa variante **segmentation d'instance**, il identifie chaque objet présent dans une image et en délimite le contour pixel par pixel via un masque coloré — contrairement à une simple boîte englobante.

Tu utiliseras ici le modèle **pré-entraîné sur COCO** (80 classes génériques), sans phase d'entraînement. L'objectif est d'intégrer correctement l'inférence dans ton pipeline Git/DVC, pas d'optimiser les performances.

> *Le notebook de correction complet est fourni séparément : correction_yolov8.ipynb*

Tâche 1 — Installer les dépendances

```
pip install ultralytics opencv-python

python -c "from ultralytics import YOLO; print('Environnement OK')"
```

Tâche 2 — Écrire le script src/segment.py

Ouvre **src/segment.py** dans VS Code et implémente le script ci-dessous. Chaque section est commentée pour en faciliter la compréhension.

> *Comportement attendu du modèle : lors de la première exécution, YOLOv8m-seg télécharge automatiquement ses poids (~52 Mo) dans le répertoire courant (racine du projet). Ce fichier yolov8m-seg.pt sera ensuite versionné via DVC à la tâche suivante.*

```
from ultralytics import YOLO

from pathlib import Path


# ■■ 1. Chemins ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

IMAGE_DIR = Path("data/images")

OUTPUT_DIR = Path("data/results")

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)


# ■■ 2. Chargement du modèle ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
# yolov8m-seg.pt est téléchargé automatiquement à la racine du projet
# lors de la première exécution (~52 Mo)
model = YOLO("yolov8m-seg.pt")

# ■■ 3. Inventaire des images ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
images = list(IMAGE_DIR.glob("*.jpg")) + list(IMAGE_DIR.glob("*.jpeg"))
print(f"Images disponibles : {len(images)}")

# ■■ 4. Inférence de segmentation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# On limite à 5 images pour ce test initial
for img_path in images[:5]:
results = model.predict(
source=str(img_path),
conf=0.25, # seuil de confiance minimal
save=True, # sauvegarde les images annotées
project=str(OUTPUT_DIR),
name="predict",
exist_ok=True
)
for r in results:
labels = [model.names[int(c)] for c in r.boxes.cls]
print(f"{img_path.name} → objets détectés : {set(labels)}")

print("Inférence terminée.")
```

Tâche 3 — Exécuter, analyser les résultats et commiter

```
# Lancer le script depuis la racine du projet
python src/segment.py
```

Par conception, Ultralytics redirige les sorties vers son propre répertoire de travail. Les images annotées ne se trouvent **pas** dans **data/results/predict/** mais dans :

```
runs\segment\predict\ # chemin réel des images annotées sous Windows
```

Ouvre ce dossier dans l'explorateur de fichiers pour visualiser les résultats : chaque image est annotée avec les masques colorés correspondant aux objets détectés.

> *Classes détectées typiquement sur des images de nourriture : bowl, cup, fork, spoon, pizza, cake, donut, hot dog. Le modèle COCO n'a pas été entraîné spécifiquement sur de la nourriture — des faux négatifs sont normaux et attendus.*

```
# Exclure les résultats d'inférence du versionnage Git (trop lourds)

"data/results/" >> .gitignore

"runs/" >> .gitignore


# Versionner le script dans Git

git add src/segment.py .gitignore

git commit -m "feat: script segmentation YOLOv8m"

git push origin main


# ■■ Bonus : versionner le modèle téléchargé via DVC ■■■■■■■■■■■■

# Le fichier yolov8m-seg.pt se trouve à la racine du projet

# On le déplace dans models/ puis on le confie à DVC

Move-Item yolov8m-seg.pt models/

dvc add models/yolov8m-seg.pt

git add models/yolov8m-seg.pt.dvc models/.gitignore

git commit -m "model: versionnage YOLOv8m-seg via DVC"

dvc push

git push origin main
```

RÉCAPITULATIF — Commandes essentielles

Git

```
git init # Initialiser un dépôt local

git status # Inspecter l'état du dépôt

git add . # Indexer toutes les modifications

git commit -m "..." # Créer un commit

git push origin main # Synchroniser vers GitHub

git pull origin main # Récupérer les mises à jour

git checkout -b nom # Créer et basculer sur une branche

git merge nom # Fusionner une branche dans main

git log --oneline # Afficher l'historique
```

DVC

```
dvc init # Initialiser DVC dans un dépôt Git

dvc add data/images # Placer un répertoire sous gestion DVC

dvc push # Transférer les données vers Google Drive

dvc pull # Récupérer les données depuis Google Drive

dvc status # Vérifier la cohérence données/pointeurs
```

Structure finale du projet

```
food-segmentation/

■■■ .dvc/ ← configuration DVC (versionné dans Git)

■■■ .gitignore

■■■ README.md

■■■ data/
```

```
■ ■■■ images/ ← 100 images (sur Drive, exclues de Git)
■ ■■■ images.dvc ← fichier pointeur DVC (dans Git)
■■■ models/
■ ■■■ yolov8m-seg.pt ← modèle (sur Drive, exclu de Git)
■ ■■■ yolov8m-seg.pt.dvc
■■■ runs/ ← résultats d'inférence (exclus de Git)
■■■ src/
■■■ segment.py ← code source (versionné dans Git)
```

Règle fondamentale : CODE → Git + GitHub | DONNÉES & MODÈLES → DVC + Google Drive